

MTGR: 美团工业级生成式推荐框架

Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang,
Xiaoguang Li, Chunzhen Jing, Yueming Han, Menglei Zhou, Lei Yu, Chuan Liu, Wei Lin

中文逐段翻译稿

摘要

缩放定律已经在自然语言处理、计算机视觉等多个领域得到了广泛验证。在推荐系统中, 近期工作采用生成式推荐以实现可扩展性, 但这类生成式方法需要放弃传统推荐模型中精心构造的交叉特征。我们发现, 这种做法会显著损害模型性能, 而且仅靠扩大模型规模根本无法弥补这一损失。为了解决该问题, 本文提出 MTGR (Meituan Generative Recommendation, 美团生成式推荐)。MTGR 基于 HSTU 架构进行建模, 并且能够保留原始深度学习推荐模型 (DLRM) 的特征, 包括交叉特征。此外, MTGR 通过用户级压缩实现训练与推理加速, 从而保证高效扩展。我们还提出了组层归一化 (Group-Layer Normalization, GLN), 以增强不同语义空间中的编码效果, 并提出动态掩码策略以避免信息泄漏。我们进一步对训练框架进行了优化, 使模型在计算复杂度达到 DLRM 的 10 到 100 倍时, 仍无需显著增加成本。与 DLRM 相比, MTGR 在单样本前向推理中实现了 65 倍 FLOPs, 在线上和线上都取得了近两年来最大的收益。这一突破已经成功部署在美团——全球最大的外卖平台——并承载主流流量。

1 引言

缩放定律已经被证明适用于大多数深度学习任务, 包括语言模型、计算机视觉以及信息检索等。本文的工作聚焦于如何在工业级推荐系统中有效地扩大排序模型规模。在工业推荐系统对于高 QPS (每秒查询数) 和低时延的要求下, 模型扩展通常同时受训练成本与推理时间的限制。当前, 排序模型的扩展研究大体可以分为两类: 深度学习推荐模型 (DLRM) 和生成式推荐模型 (GRM)。DLRM 通过对单个用户-物品对建模来学习兴趣概率, 并通过构造更复杂的映射实现扩展; GRM 则像自然语言一样按 token 组织数据, 并通过 Transformer 架构执行下一个 token 预测。

在工业推荐系统中, DLRM 已经使用了近十年, 其输入通常包含大量设计精细的手工特征, 例如交叉特征, 以提升模型性能。然而, 当面对扩展需求时, DLRM 存在两个显著缺陷。其一, 随着用户行为呈指数式增长, 传统 DLRM 无法高效处理完整用户行为, 往往只能借助序列检索或设计低复杂度模块来学习, 这限制了模型的学习能力。其二, 基于 DLRM 的扩展, 其训练和推理成本会随着候选项数量近似线性增长, 导致代价高得难以承

受。

对于 GRM，近期研究指出其具有优异的可扩展性。我们认为有两个关键因素：其一，GRM 直接对完整的用户行为链进行建模，将同一用户下多个曝光样本压缩为一个样本，这显著减少了计算冗余，同时相较于 DLRM 能够端到端编码更长的序列；其二，GRM 采用带有高效注意力计算的 Transformer 架构，使模型的训练与推理能够满足工业推荐系统的要求。然而，GRM 高度依赖下一个 token 预测来建模完整用户行为序列，这要求移除候选项与用户之间的交叉特征。我们发现，去除交叉特征会严重损害模型性能，而且这种退化完全无法通过扩大模型规模来弥补。

如何构建一种既能利用交叉特征以保证效果、又具备 GRM 可扩展性的排序模型？为了解决这一问题，我们提出了美团生成式推荐框架 MTGR。与传统 DLRM 和 GRM 相比，MTGR 取其所长、避其所短。MTGR 保持与传统 DLRM 一致的输入特征，包括交叉特征。具体而言，MTGR 通过将用户特征和候选特征转换为不同的 token 来重组特征，从而形成适合高效扩展的 token 序列。随后，MTGR 将交叉特征融入候选 token 中，并采用判别式损失进行学习。

MTGR 采用与文献 [23] 中相似的 HSTU (Hierarchical Sequential Transduction Units) 架构进行建模。在 HSTU 中，我们提出组层归一化 (GLN)，对不同类型的 token 分别归一化，从而使模型能够更好地同时建模多种异构信息。此外，我们提出动态掩码策略，结合全注意力、自回归和仅对自身可见三种方式，以兼顾性能并避免信息泄漏。

不同于工业界常用的 TensorFlow，MTGR 的训练框架构建在 TorchRec 之上，并针对计算效率进行了优化。具体而言，为了支持稀疏 embedding 条目的实时插入与删除，我们使用动态哈希表替代静态表。为了提高效率，我们引入动态序列均衡以解决 GPU 之间的计算负载不平衡问题，并采用 embedding ID 去重与自动表合并以加速 embedding 查找。我们还引入混合精度训练与算子融合等实现层面的优化。与原始 TorchRec 相比，我们优化后的框架在 100 多张 GPU 上运行时，训练吞吐提升了 1.6 倍到 2.4 倍，并且具有良好的扩展性。

我们首先在一个小规模数据集上验证了 MTGR 的可扩展性。随后，我们设计了三种不同规模的模型，并基于超过六个月的数据进行训练，以验证离线和在线性能上的缩放定律。与经过多年优化的 DLRM 基线相比，大模型版本在单样本前向计算上达到了 65 倍 FLOPs，并在我们的业务场景中将转化量提升了 1.22%，将点击率 (CTR) 提升了 1.31%。同时，训练成本保持不变，推理成本还降低了 12%。MTGR-large 已经部署在美团外卖推荐系统中，为数亿用户提供服务。

总的来说，本文的贡献如下：

- MTGR 结合了 DLRM 与 GRM 的优势，在保留 DLRM 全部特征（包括交叉特征）的同时，也具备与 GRM 相当的可扩展性。
- 我们提出了组层归一化与动态掩码策略，以获得更好的模型性能。
- 我们对基于 TorchRec 的 MTGR 训练框架进行了系统性优化，以提升训练效率。

- 我们通过离线与在线实验展示了 MTGR 的性能与计算复杂度之间的幂律关系，以及其相对于 DLRM 的优势。

2 相关工作

2.1 深度学习推荐模型

经典的 DLRM 结构通常包含许多输入，例如上下文信息（如时间、位置）、用户画像（如性别、年龄）、用户行为序列以及目标物品，同时还会使用大量交叉特征。在排序模型中，两个尤其重要的模块是行为序列处理和特征交互学习。行为序列模块通常采用目标注意力机制，以捕获用户历史行为与待估计物品之间的相似性。特征交互模块则旨在捕获不同特征之间（包括用户和物品之间）的相互作用，以生成最终预测结果。

2.2 推荐模型扩展

针对 DLRM 中不同的可扩展模块，已有两种不同思路。一种是扩展交叉模块，即扩展融合用户与物品信息的特征交互模块。文献 [25] 引入了可堆叠的 Wukong 层来进行扩展；文献 [6] 则使用多 embedding 策略来缓解 embedding collapse 问题，从而增强模型的可扩展性。另一种是扩展用户模块，即只扩展用户侧部分，这样在推理阶段会更加友好。文献 [7,26] 通过仅扩展用户表示并将其缓存或广播到待估计的多个物品上，从而降低在线推理成本；文献 [16,22] 则为用户表示设计了预训练方法，并证明了其在下游任务中的可扩展性。

与 DLRM 相对应的是 GRM。文献 [23] 通过 HSTU 验证了扩展到万亿参数规模时的缩放定律；文献 [3] 则使用语义编码替代传统 ID 表示，并结合 DPO 优化与基于 Transformer 的框架，用统一的生成式模型替代级联式学习框架。

3 预备知识

3.1 数据组织方式

传统上，对于一个用户及其对应的 K 个候选项，第 i 个用户-候选项样本可表示为 $D_i = [U, \vec{S}, \vec{R}, C_i, I_i]$ 。其中， $U = [U_1, \dots, U_{N_U}]$ 表示用户画像特征（如年龄、性别等），每个特征 U_i 都是一个标量， N_U 表示所用特征数量。 $\vec{S} = [S_1, \dots, S_{N_S}]$ 包含用户历史交互过的物品序列。每个元素 $S_i = [s_1, \dots, s_{N_s}]$ 表示一个物品，它由若干被选取的特征（如物品 ID、标签、物品平均 CTR 等）组成。类似于 $\vec{S}$ ， $\vec{R}$ 记录距离当前请求最近的、发生在数小时或一天内的交互行为，表示用户的实时动作和偏好，并与 $\vec{S}$ 共享相同的特征。 $C = [C_1, \dots, C_{N_C}]$ 由用户与候选项之间的交叉特征组成。 $I = [I_1, \dots, I_{N_I}]$ 则包含候选项的特征，如物品 ID、标签和品牌等。 I 与候选项相关，并在不同用户之间共享。

图 1: 传统排序模型中的数据组织方式与工作流程。图中给出了一个简化特征示例： U 包

含“年龄”和“性别”； $\vec{S}$ 与 $\vec{R}$ 分别由两个物品组成，每个物品具有“ID”“标签”和“品牌”； C 包含“ctr”和“pv”，表示用户针对目标候选选项的历史点击率以及曝光次数，而该候选选项具有“ID”“标签”和“品牌”等特征。

3.2 推荐系统中的排序模型

对于输入样本 D ，传统推荐系统会独立处理每个样本。具体而言，首先对 D 中的特征进行 embedding，将样本转换为稠密表示。形式化地说， U, C, I 中的特征经过 embedding 并拼接后，分别得到 $\text{Emb}_U \in \mathbb{R}^{K \times d_U}$ 、 $\text{Emb}_C \in \mathbb{R}^{K \times d_C}$ 和 $\text{Emb}_I \in \mathbb{R}^{K \times d_I}$ 。

对于 $\vec{S}$ 与 $\vec{R}$ 中的特征，每个物品 S_i 的各特征也会类似地被 embedding 并拼接为 $\text{Emb}_{S_i} \in \mathbb{R}^{d_s}$ ，然后 $\vec{S}$ 中各物品沿另一维拼接，得到 $\text{Emb}_{\vec{S}} \in \mathbb{R}^{N_{\vec{S}} \times d_s}$ 。

为了提取历史交互物品与候选选项之间的用户兴趣，通常会使用目标注意力机制，其中以目标为 query，以序列特征为 key/value。形式化表示为：

$$\text{Feat}_{\vec{S}} = \text{Attention}(\text{Emb}_I, \text{Emb}_{\vec{S}}, \text{Emb}_{\vec{S}}) \in \mathbb{R}^{K \times d_s}. \quad (1)$$

式 (1) 按照候选选项 I 对 $\vec{S}$ 进行聚合。最终，来自 D 的嵌入特征与经过处理的特征会拼接表示为：

$$\text{Feat}_D = [\text{Emb}_U, \text{Feat}_{\vec{S}}, \text{Feat}_{\vec{R}}, \text{Emb}_C, \text{Emb}_I] \in \mathbb{R}^{K \times (d_U + d_s + d_C + d_I)}. \quad (2)$$

随后， Feat_D 会被送入多层感知机 (MLP) 中，为每个样本输出 logit。这个 logit 在训练阶段用于学习，在推理阶段用于排序。

图 1 展示了传统排序模型下一个简化的数据组织方式和工作流程。首先对这些特征进行 embedding，随后对不同种类的 embedding 采用不同方式处理，最后将处理后的特征拼接，并通过 MLP 进行特征交互，为每个候选选项生成最终的 logit。

3.3 推荐系统中的扩展困境

模型扩展一直是提升排序性能的常见方法。一般而言，模型扩展主要面向用户模块和交叉模块。用户模块负责处理用户特征，包括序列特征，并生成用户相关表示。扩展用户模块能够得到更好的用户表示。此外，由于同一个用户只需推理一次，其结果可供所有候选选项共享，因此即使用户模块计算量较大，也不会给系统带来难以承受的负担。然而，仅扩展用户模块并不能直接增强用户与物品之间的特征交互。

相反，另一类方法旨在扩展交叉模块，即特征拼接之后的特征交互 MLP。这类方法通过更加关注用户与候选选项之间的交互来增强排序能力。然而，由于交叉模块需要对每个候选选项分别执行推理，其计算开销会随着候选选项数量线性增长，从而带来不可接受的系统时延。

传统推荐系统中的这种扩展困境，迫切需要一种新的扩展方式：既能实现用户与候选选项之间的高效特征交互，同时又能使推理成本相对于候选选项数量呈次线性增长。MTGR 通

过数据重组及相应的架构优化，创新了推荐系统中的扩展方式。

4 MTGR 的数据重组与架构

4.1 面向训练与推理效率的用户样本聚合

与第 3.1 节中的特征分类方式不同，对于候选项中的第 i 个样本，MTGR 将特征组织为 $D_i = [U, \vec{S}, \vec{R}, [C_i, I_i]]$ 。具体而言，交叉特征 C 被视为候选项特征的一部分。在 MTGR 中，训练阶段按照用户在特定时间窗口内对候选项进行聚合，推理阶段则按请求进行聚合。由于聚合来自同一用户，因此聚合后的样本可以共享同一份用户表示 $(U, \vec{S}, \vec{R})$ 。其中， $\vec{R}$ 将用户在另一特定窗口内的全部实时交互物品按交互时间顺序排列。

图 2(a) 展示了这种聚合方式：与图 1 中一次只预测一个候选项不同，图 2(a) 将三个候选项聚合为一个样本，从而复用同一份用户表示。形式化地，它构成如下同一用户的特征表示：

$$D = [U, \vec{S}, \vec{R}, [C, I]_1, \dots, [C, I]_K]. \quad (3)$$

通过将候选项聚合进同一个样本，MTGR 只需进行一次计算，就能为全部候选项输出分数，从而节省大量资源。尤其是在训练中，用户级聚合过程将训练样本数从“所有用户对应的所有候选项规模”大幅减少下来。对于推理过程，单次请求中的候选项也按照上述方式分组，因此 MTGR 只需执行一次推理即可完成全部候选项排序，而不再需要按候选项个数分别推理。这种聚合消除了推理成本对候选项数量的依赖，为模型扩展留下了空间和潜力。

式 (3) 同时包含标量特征和序列特征。为了统一输入格式，MTGR 将特征与序列都转换为 token。对于 U 中的标量特征，每个特征天然可转换为一个独立 token，维度记为 $\text{Feat}_U \in \mathbb{R}^{N_U \times d_{\text{model}}}$ ，其中 d_{model} 是为所有 token 设定的统一维度。对于 $\vec{S}$ 和 $\vec{R}$ 中的序列特征，每个物品 S 被视为一个 token。其内部特征首先经过 embedding 并拼接，再通过一个 MLP 模块统一维度。形式化地：

$$\text{Feat}_{S_i} = \text{MLP}(\text{Concat}(\text{Emb}_s)) \in \mathbb{R}^{d_{\text{model}}}. \quad (4)$$

随后，序列中各个 S 的特征沿另一维拼接，得到 $\text{Feat}_{\vec{S}} \in \mathbb{R}^{N_{\vec{S}} \times d_{\text{model}}}$ 。

类似地，候选项中的每个物品 I 也被视为一个 token。候选项中的特征经过 embedding 与拼接后，再通过另一个 MLP 统一到相同维度。所有候选项随后被拼接为一串 token：

$$\text{Feat}_I = \text{Concat}(\text{MLP}(\text{Concat}(\text{Emb}_{C_i}, \text{Emb}_{I_i}))). \quad (5)$$

最终，将来自 $U, \vec{S}, \vec{R}, [C, I]$ 的所有 token 连接，形成一个长 token 序列：

$$\text{Feat}_D = \text{Concat}([\text{Feat}_U, \text{Feat}_{\vec{S}}, \text{Feat}_{\vec{R}}, \text{Feat}_I]) \in \mathbb{R}^{(N_U + N_{\vec{S}} + N_{\vec{R}} + N_I) \times d_{\text{model}}}. \quad (6)$$

图 2: MTGR 的数据组织方式与模型架构。(a) 展示了 MTGR 的数据重组与整体流程: 将 3 个候选项的特征与同一个用户进行聚合。随后, 这些特征经过 *embedding* 并通过 MLP 转换为 *token*, 组成一段输入序列, 并在带掩码的自注意力中进行建模。候选项 *token* 的表示再经过另一个 MLP 输出 *logit*。(b) 给出了由 L 层组成的自注意力模块细节: 输入序列首先经过组层归一化以及 4 个投影 ($Q/K/V/U$); 随后在定制掩码下执行自注意力; 完成 *value update* 之后, 再将更新后的 V 与投影后的 U 做点乘; 最后, 更新值再次经过分组层归一化, 并与残差连接相加。(c) 展示了用于避免信息泄漏的定制掩码, 其中 “*rt*” 和 “*target*” 按从近到远的时间顺序排列: 用户特征 ($U, \vec{S}$) 对所有 *token* 可见; “*rt*” 仅对其后出现的 *token* 可见; “*target*” 仅对自身可见。

4.2 统一 HSTU 编码器

来自同一用户的样本被聚合为一个 *token* 序列, 因此天然适合使用自注意力进行处理。受 HSTU 启发, MTGR 采用由多层自注意力构成的仅编码器 (encoder-only) 架构进行建模。

与大语言模型类似, 输入 *token* 序列按层进行处理。如图 2 所示, 在自注意力模块中, 输入序列 X 首先经过组层归一化。来自相同域的特征 (例如 U) 构成一个组。组层归一化能够在自注意力之前, 使不同域的 *token* 共享相近的分布, 并对齐不同域所对应的语义空间, 即 $\tilde{X} = \text{GroupLN}(X)$ 。然后, 对归一化后的输入进行 4 组不同投影, 得到 $K, Q, V, U = \text{MLP}_{K/Q/V/U}(\tilde{X})$ 。其中, Q 和 K 用于带有 SiLU 非线性激活的多头注意力计算, 且得到的注意力分数会除以输入特征总长度作为平均化因子。接着, 在注意力分数上施加定制掩码 M , 并利用投影后的 V 进行 *value update*:

$$\tilde{V} = \frac{\text{silu}(K^T Q)}{(N_U + N_{\vec{S}} + N_{\vec{R}} + N_I)} MV. \quad (7)$$

随后, 将投影后的 U 与更新后的 $\tilde{V}$ 做点乘, 并再次施加组层归一化。最后, 加入残差模块, 并在其上方再接一个 MLP:

$$X = \text{MLP}(\text{GroupLN}(\tilde{V} \odot U)) + X. \quad (8)$$

动态掩码 [23] 使用因果掩码来进行序列建模。然而, 这种实现方式在 MTGR 中并未带来显著提升。此外, 由于 $\vec{R}$ 记录的是用户最近的交互, 而这些交互的时间可能与样本聚合窗口重合, 因此如果在 MTGR 中直接使用简单的因果掩码, 可能会造成信息泄漏。例如, 晚上发生的交互不应暴露给下午的候选项, 但在聚合后它们可能出现在同一样本中。这一困境要求我们设计一种更灵活且高效的掩码机制。

在 MTGR 中, $U, \vec{S}$ 被视为静态信息 (下文将 $U, \vec{S}$ 记为 “静态序列”), 因为它们的信息来自聚合窗口之前, 不会引发因果性错误。 $\vec{R}$ 则是动态的, 因为它会实时逐步纳入用户的交互信息。因此, MTGR 对静态序列使用全注意力, 对 $\vec{R}$ 使用带动态掩码的自回归注意力, 而对候选项之间使用对角掩码。具体而言, MTGR 的掩码遵循如下三条规则:

- 静态序列对所有 token 都可见；
- 动态序列的可见性服从因果性，即每个 token 只对其之后出现的 token 可见，这其中也包括候选项 token；
- 候选项 token (C, I) 仅对其自身可见。

图 2(c) 给出了一个动态掩码示例：“age”“ctr”表示来自 U 的特征 token；“seq1”“seq2”表示 $\vec{S}$ ；“rt1”“rt2”表示 $\vec{R}$ ；“target1”到“target3”表示候选项。矩阵中某一行的白色方块表示该 token 可以使用其他 token 的信息，而列则表示该 token 是否对其他 token 可见。对于 U 和 S ，使用全注意力，因此从“age”到“seq2”形成一个白色方块区域。对于“rt1”到“rt2”，假设“rt1”比“rt2”出现得更晚，因此其上三角区域为白色，表示“rt1”可以使用“rt2”的信息，而“rt2”不能看到“rt1”。此外，假设“target2”和“target3”比“rt1”出现得更早，因此“rt1”对它们不可见。再进一步，假设“rt2”比“target1”和“target2”更早，但比“target3”更晚，因此“rt2”对“target3”不可见，而“target3”由于出现在所有“rt”之前，也无法使用来自“rt”的信息。

5 训练系统

为了方便地进行 MTGR 模型结构设计与开发，并能够更便捷地吸收快速发展的 LLM 社区成果，我们决定不再沿用此前基于 TensorFlow 的训练框架，而是在 PyTorch 生态中重构训练系统。具体而言，我们在 TorchRec 的基础上进行了功能扩展与定向优化，使其更适配 MTGR 的模型特点，并最终实现了 MTGR 的高效训练。与 TorchRec 相比，我们优化后的框架在 100 多张 GPU 上运行时，训练吞吐可提升 1.6 倍到 2.4 倍。与 DLRM 基线相比，我们在单样本前向计算上实现了 65 倍 FLOPs，而训练成本几乎保持不变。下面介绍若干关键优化。

动态哈希表。 TorchRec 使用固定大小的表来处理稀疏 embedding，这并不适合大规模工业流式训练场景。首先，当静态表达到预设容量后，无法为新增用户和物品实时分配 embedding；其次，为了避免 ID 溢出，静态 embedding 表往往需要预留比实际需求更大的空间，从而导致内存资源利用低效。为了解决这些问题，我们开发了基于哈希技术的高性能 embedding 表，能够在训练期间为稀疏 ID 动态分配空间。其设计采用了解耦哈希表架构，将 key 存储与 value 存储分离。key 存储提供轻量级映射系统，将 key 映射到指针；这些指针指向 embedding 向量；而 value 存储则保存 embedding 向量及额外元数据（如计数器和时间戳），用于支持淘汰策略。该两部分系统实现了两个主要目标：一是仅通过复制 key 存储而非大规模 embedding 本体，即可动态扩容；二是通过紧凑方式组织 key，提高 key 扫描效率。

Embedding 查找。 embedding lookup 过程采用 All-to-all 通信进行跨设备 embedding 交换。为了尽量减少设备之间重复 ID 的传输，我们实现了一个两阶段流程，以保证 ID 在通信前后都保持唯一。

负载均衡。在推荐系统中，用户行为序列往往呈现长尾分布，只有少数用户拥有很长的序列，而大多数用户的序列较短。这使得在固定 batch size (简称 BS) 训练时，很容易产生显著的计算负载不平衡。常见做法是使用序列打包技术，将多个短序列拼接为更长序列。然而，这种方法要求仔细调整掩码，以防止不同序列在注意力计算中相互干扰，实现代价较高。我们的简单方案是引入动态 BS：根据输入数据的实际序列长度动态调整每张 GPU 的本地 BS，从而使各 GPU 的计算负载保持相近。此外，我们还调整了梯度聚合策略，使每张 GPU 的梯度按照其 BS 加权，从而保证与固定 BS 情形下的计算逻辑一致。

其他优化。为进一步提升训练效率，我们采用三条不同 stream 的流水线技术：copy、dispatch 和 compute。copy stream 负责将输入数据从 CPU 传输到 GPU；dispatch stream 负责利用 ID 执行表查找；compute stream 则负责前向计算和反向更新。例如，当 compute stream 正在执行 batch T 的前向与反向计算时，copy stream 会同时加载 batch $T + 1$ ，从而尽量降低 I/O 延迟。当 batch T 的反向更新结束后，dispatch stream 会立刻启动 batch $T + 1$ 的表查找与通信。此外，我们还采用了 bf16 混合精度训练，并基于 cutlass 设计了专用注意力 kernel 以加速训练过程。

6 实验

6.1 实验设置

数据集。公共数据集通常使用相互独立的 ID 与属性特征，而很少显式引入交叉特征。然而，在真实应用中，交叉特征十分重要。在我们的场景中，交叉特征是一类关键特征，且通常经过精心人工设计，例如用户-物品交互、用户与高层类别交互、物品与时空信息交互等。为了弥补公共数据集中交叉特征缺失的问题，我们基于美团真实工业级推荐系统日志构建了训练数据集。与公开数据集不同，我们的真实数据集具有更丰富的交叉特征集合和更长的用户行为序列。使用工业级数据进行实验，能够更好地体现这些交叉特征在真实推荐系统中的重要作用。此外，由于数据规模足够大，复杂模型也能在训练中获得更充分的收敛。对于离线实验，我们收集了连续 10 天的数据，其统计信息如表 1 所示。对于在线实验，为了与已经训练超过两年的 DLRM 基线对比，我们进一步构建了一个跨度超过 6 个月的长期数据集。

表 1: 数据集统计信息

数据集	用户数	物品数	曝光数	点击数	购买数
训练集	0.21 billion	4,302,391	23.74 billion	1.08 billion	0.18 billion
测试集	3,021,198	3,141,997	76,855,608	4,545,386	769,534

基线模型。对于 DLRM，我们比较了两种序列建模方式：基于序列检索的 SIM，以及针对原始长序列进行端到端建模的 E2E。在扩展方式上，我们比较了 DNN、MoE、Wukong、MultiEmbed 和 UserTower。MoE 使用 4 个 expert，每个 expert 都具有与基础 DNN 相同复杂度的网络。Wukong 和 MultiEmbed 的计算复杂度与 MoE 保持相同。UserTower 使用

一组可学习 query，并在用户行为上插入 qFormer 层以及另一个 MoE (16 个 expert) 模块。UserTower 的计算复杂度是 MoE 的 3 倍，但由于同一用户的多个待预测物品可以共享这部分计算，因此其推理成本较低，并在我们的场景中取得了良好效果。

MTGR 使用 E2E 方式处理全部序列信息。此外，如表 2 所示，我们设置了三种不同规模的 MTGR 模型，以验证其可扩展性。

表 2: 不同模型设置及其计算复杂度对比

模型	设置	学习率	GFLOPs/样本
UserTower-SIM	-	8×10^{-4}	0.86
MTGR-small	$n_{\text{layer}} = 3, d_{\text{model}} = 512, n_{\text{heads}} = 2$	3×10^{-4}	5.47
MTGR-medium	$n_{\text{layer}} = 5, d_{\text{model}} = 768, n_{\text{heads}} = 3$	3×10^{-4}	18.59
MTGR-large	$n_{\text{layer}} = 15, d_{\text{model}} = 768, n_{\text{heads}} = 3$	1×10^{-4}	55.76

评估指标。在线下实验中，我们关注 CTR 和 CTCVR (Click-Through Conversion Rate) 两个任务，并使用 AUC 和 GAUC (Group AUC) 进行评估。GAUC 是对用户维度下各个 AUC 的平均，相较于 AUC，它更加关注模型对同一用户内部候选项的排序能力。在线上实验中，我们关注两个指标：PV_CTR (每次页面浏览的 CTR) 和 UV_CTCVR (每个用户访问的 CTCVR)，其中 UV_CTCVR 是衡量业务增长最关键的指标。

参数设置。模型采用 Adam 优化器训练。对于 DLRM，每张 GPU 的 batch size 为 2400，使用 8 张 NVIDIA A100 GPU 训练；对于 MTGR，batch size 设为 96，使用 16 张 NVIDIA A100 GPU 训练。如表 2 所示，随着模型复杂度增加，学习率会相应降低。此外，随着计算复杂度增长，我们也通过配置不同的 embedding 维度按比例扩大稀疏参数规模。若一个 token 由 k 个特征组成，则每个特征的 embedding 维度通常设置为接近 d_{model}/k 的整数。值得注意的是，为了避免稀疏参数过度膨胀所带来的额外开销，我们主要增加基数较小的稀疏特征维度，而对极稀疏特征维度保持不变。最后， $\vec{S}$ 的最大长度设为 1000， $\vec{R}$ 的最大长度设为 100。

6.2 整体性能比较

我们在 10 天数据集上评估了 MTGR 与其他基线方法的性能。表 3 给出了不同模型的表现。不同模型在各个离线指标上的差异具有较高一致性。根据以往经验，在我们的场景中，离线指标提升 0.001 就已经具有显著意义。在多种 DLRM 版本中，Wukong-SIM 和 MultiEmbed-SIM 的效果优于 MoE-SIM；UserTower-SIM 表现最好，而 UserTower-E2E 相比 UserTower-SIM 略有下降。我们推测，在 DLRM 范式下，模型复杂度仍不足以建模全部序列信息，因此出现了欠拟合。相比之下，我们提出的 MTGR 即便是最小版本，也超过了最强的 DLRM 模型；并且三种不同规模的 MTGR 模型都表现出良好的可扩展性，其性能会随着模型复杂度提升而平滑增长。

表 3: 整体性能对比。Impr.% 表示最佳 MTGR 模型相对于最强 DLRM 基线的相对提升。

模型	CTR AUC	CTR GAUC	CTCVR AUC	CTCVR GAUC
DNN-SIM	0.7432	0.6679	0.8737	0.6504
MoE-SIM	0.7484	0.6698	0.8750	0.6519
MultiEmbed-SIM	0.7501	0.6715	0.8766	0.6525
Wukong-SIM	0.7568	0.6759	0.8800	0.6530
UserTower-SIM	0.7593	0.6792	0.8815	0.6550
UserTower-E2E	0.7576	0.6787	0.8818	0.6548
MTGR-small	0.7631	0.6826	0.8840	0.6603
MTGR-medium	0.7645	0.6843	0.8849	0.6625
MTGR-large	0.7661	0.6865	0.8862	0.6646
Impr.%	0.8956	1.0748	0.4990	1.4656

6.3 消融实验

我们在 MTGR-small 上对 MTGR 的两个组件进行消融:动态掩码和组层归一化(GLN)。消融结果如表 4 所示。去掉任一组件都会使性能显著下降,其下降幅度与从 MTGR-small 提升到 MTGR-medium 时的增幅相近。这说明动态掩码和 GLN 对 MTGR 都十分重要。此外,我们还额外验证了交叉特征对 MTGR 的重要性。当移除交叉特征后,性能指标大幅下滑,甚至抹去了 MTGR-large 相对 DLRM 的全部优势,这进一步说明交叉特征在真实推荐系统中的关键作用。

表 4: MTGR 的消融实验

模型	CTR AUC	CTR GAUC	CTCVR AUC	CTCVR GAUC
MTGR-small	0.7631	0.6826	0.8840	0.6603
去除交叉特征	0.7495	0.6689	0.8736	0.6514
去除 GLN	0.7606	0.6809	0.8826	0.6585
去除动态掩码	0.7620	0.6810	0.8828	0.6587

6.4 可扩展性

图 3 展示了 MTGR 的可扩展性。我们基于 MTGR-small, 分别围绕三个超参数进行测试: HSTU block 数量、 d_{model} 和输入序列长度。可以看到, MTGR 在不同超参数维度上都表现出良好的可扩展性。此外, 图 3(d) 还展示了模型性能与计算复杂度之间的幂律关系。纵轴表示相对于最佳 DLRM 模型 UserTower-SIM 而言, CTCVR GAUC 指标的增益; 横轴则表示相对于 UserTower-SIM 的计算复杂度对数倍数。

图 3: 随着 HSTU block 数、 d_{model} 和训练序列长度的增加, MTGR 的性能平滑提升。

6.5 在线实验

为了进一步验证 MTGR 的有效性，我们在美团外卖平台上部署了 MTGR，并以 2% 流量进行 AB 测试。实验流量每天覆盖数百万次曝光，这体现了我们对实验的信心。对比基线为线上最先进的 DLRM 模型 UserTower-SIM，该模型已经持续学习了 2 年。我们使用最近 6 个月的数据训练 MTGR 模型，并将其部署上线进行比较。

尽管 MTGR 的训练数据量显著低于 DLRM 对应模型，其离线和在线指标仍然大幅超过 DLRM 基线。如表 5 所示，无论离线还是在线指标，都表现出明显的可扩展性。我们还发现，随着训练 token 数量增加，相对于 DLRM 的优势还会继续扩大。最终，在 CTCVR GAUC 指标上，我们的大模型版本甚至超过了过去一年所有优化累计带来的增益。

该模型已经在我们的场景中全面部署，其训练成本与 DLRM 相当，而推理成本降低了 12%。对于 DLRM，其推理成本大致与候选项数量线性相关；而 MTGR 通过对单次请求中的所有候选项执行用户级聚合，使推理成本随候选项数量呈次线性增长。这帮助我们降低了线上推理开销。

表 5: 不同版本 MTGR 的离线与在线效果比较

模型	CTR GAUC 提升	CTCVR GAUC 提升	PV_CTR 提升	UV_CTCVR 提升
MTGR-small	+0.0036	+0.0154	+1.04%	+0.04%
MTGR-medium	+0.0071	+0.0182	+2.29%	+0.62%
MTGR-large	+0.0153	+0.0288	+1.90%	+1.02%

7 结论

本文提出了 MTGR，这是一种基于 HSTU、用于探索推荐系统中缩放定律的新型排序框架。MTGR 结合了 DLRM 与 GRM 的优势，既可以使用交叉特征来保证模型性能，又具备与 GRM 相当的可扩展性。MTGR 已经在我们的业务场景中部署，并取得了显著收益。未来，我们将探索如何将 MTGR 扩展到类似大语言模型的多场景建模中，以构建具备广泛知识的推荐基础模型。

参考文献

参考文献条目较多，且主要用于索引原始文献来源，故本翻译稿中保留其原始英文信息，不再逐条翻译标题。读者可对照原文 PDF 查阅完整条目。

1. Qiwei Chen, Changhua Pei, Shanshan Lv, Chao Li, Junfeng Ge, and Wenwu Ou. 2021. End-to-end user behavior retrieval in click-through rate prediction model. arXiv preprint arXiv:2108.04468 (2021).

2. Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Re. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
3. Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. *arXiv preprint arXiv:2502.18965* (2025).
4. Yan Fang, Jingtao Zhan, Qingyao Ai, Jiaxin Mao, Weihang Su, Jia Chen, and Yiqun Liu. 2024. Scaling laws for dense retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1339–1349.
5. Cesar Ferri, Jose Hernandez-Orallo, and Peter A Flach. 2011. A coherent interpretation of AUC as a measure of aggregated classification performance. In *Proceedings of the 28th International Conference on Machine Learning*. 657–664.
6. Xingzhuo Guo, Junwei Pan, Ximei Wang, Baixu Chen, Jie Jiang, and Mingsheng Long. 2023. On the embedding collapse when scaling up recommendation models. *arXiv preprint arXiv:2310.04400* (2023).
7. Ruidong Han, Qianzhong Li, He Jiang, Rui Li, Yurou Zhao, Xiang Li, and Wei Lin. 2024. Enhancing CTR Prediction through Sequential Recommendation Pre-training: Introducing the SRP4CTR Framework. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 3777–3781.
8. Dmytro Ivchenko, Dennis Van Der Staay, Colin Taylor, Xing Liu, Will Feng, Rahul Kindi, Anirudh Sudarshan, and Shahin Sefati. 2022. TorchRec: a pytorch domain library for recommendation systems. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 482–483.
9. Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
10. Mario Michael Krell, Matej Kosec, Sergio P Perez, and Andrew Fitzgibbon. 2021. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv preprint arXiv:2107.02027* (2021).
11. Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*. PMLR, 19730–19742.
12. Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xDeepFM: Combining explicit and implicit feature interactions for recom-

- mender systems. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 1754–1763.
13. Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 1930–1939.
 14. William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision. 4195–4205.
 15. Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In Proceedings of the 29th ACM International Conference on Information & Knowledge Management. 2685–2692.
 16. Kyuyong Shin, Hanock Kwak, Su Young Kim, Max Nihlen Ramstrom, Jisu Jeong, Jung-Woo Ha, and Kyung-Min Kim. 2023. Scaling law for recommendation models: Towards general-purpose user representations. In Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37. 4596–4604.
 17. Zihua Si, Lin Guan, ZhongXiang Sun, Xiaoxue Zang, Jing Lu, Yiqun Hui, Xingchao Cao, Zeyu Yang, Yichen Zheng, Dewei Leng, et al. 2024. Twin v2: Scaling ultra-long user behavior sequence modeling for enhanced ctr prediction at kuaishou. In Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. 4890–4897.
 18. Hongyan Tang, Junning Liu, Ming Zhao, and Xudong Gong. 2020. Progressive layered extraction (PLE): A novel multi-task learning model for personalized recommendations. In Proceedings of the 14th ACM Conference on Recommender Systems. 269–278.
 19. Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. DCN v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In Proceedings of the Web Conference 2021. 1785–1797.
 20. Xu Wang, Jiangxia Cao, Zhiyi Fu, Kun Gai, and Guorui Zhou. 2024. HoME: Hierarchy of Multi-Gate Experts for Multi-Task Learning at Kuaishou. arXiv preprint arXiv:2408.05430 (2024).
 21. Yuxiang Wang, Xiao Yan, Chi Ma, Mincong Huang, Xiaoguang Li, Lei Yu, Chuan Liu, Ruidong Han, He Jiang, Bin Yin, et al. 2025. MTGRBoost: Boosting Large-scale Generative Recommendation Models in Meituan. arXiv preprint arXiv:2505.12663 (2025).

22. Bencheng Yan, Shilei Liu, Zhiyuan Zeng, Zihao Wang, Yizhen Zhang, Yujin Yuan, Langming Liu, Jiaqi Liu, Di Wang, Wenbo Su, et al. 2025. Unlocking Scaling Law in Industrial Recommendation Systems with a Three-step Paradigm based Large User Model. arXiv preprint arXiv:2502.08309 (2025).
23. Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. arXiv preprint arXiv:2402.17152 (2024).
24. Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. 2022. Scaling vision transformers. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 12104–12113.
25. Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. 2024. Wukong: Towards a scaling law for large-scale recommendation. arXiv preprint arXiv:2403.02545 (2024).
26. Wei Zhang, Dai Li, Chen Liang, Fang Zhou, Zhongke Zhang, Xuwei Wang, Ru Li, Yi Zhou, Yaning Huang, Dong Liang, et al. 2024. Scaling User Modeling: Large-scale Online User Representations for Ads Personalization in Meta. In Companion Proceedings of the ACM Web Conference 2024. 47–55.
27. Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 1059–1068.